

目录

一、 串行代码解读	1
(一) 数据结构.	1
1. 二元产生式.	1
2. 一元产生式.	1
3. 产生式索引.	1
4. 语法子树	2
(二) 算法理解.	2
1. 算法核心过程	2
2. 数据依赖	4
(三) 算法实现.	4
1. 主要执行流程	4
2. 输入	5
二、 并行程序设计与实现	5
(一) 并行程序设计.	5
(二) 并行程序实现.	6
三、 实验	6
(一) 实验环境.	7
(二) 实验组别.	7
1. 串行执行	7
2. 并行区域位于外层循环内.	7
3. 默认调度策略	7
4. 动态调度策略	7

5. 默认引导式调度.....	8
(三) 实验数据记录表.....	8
四、 问题的回答.....	9
(一) 如何将任务进行分块的?	9
(二) 如何提升运行速度与增强程序扩展性?	9
(三) 如何测量程序运行时间?	10
(四) 比较图	10
(五) 加速比变化的趋势.....	11
(六) 最佳方案.....	12

一、串行代码解读

（一）数据结构

该程序中声明了数种结构体，定义了结构体类型的多个变量，以存储文法的相关信息和 CYK 算法的数据。

1. 二元产生式

二元产生式结构体 `Production2`，其声明代码如下。

```
1 struct Production2 {  
2     int parent;  
3     int child1;  
4     int child2;  
5 } production2[MAX_PRODUCTION2_NUM];
```

二元产生式声明与定义

这段代码也同时定义了该结构体类型的数组变量 `production2`，用于存储 Chomsky 范式中的二元产生式。其中 `parent` 表示产生式的父节点（非终结符），`child1` 表示产生式的第一个子节点（非终结符），`child2` 表示产生式的第二个子节点（非终结符）。

2. 一元产生式

一元产生式结构体 `Production1`，其声明代码如下。

```
1 struct Production1 {  
2     int parent;  
3     char child;  
4 } production1[MAX_PRODUCTION1_NUM];
```

一元产生式声明与定义

这段代码也同时定义了该结构体类型的数组变量 `production1`，用于存储 Chomsky 范式中的一元产生式。`parent` 表示产生式的父节点（非终结符），`child` 表示产生式的子节点（终结符）。

3. 产生式索引

算法在搜索产生式时，使用的是产生式右侧的符号组合，查找是否有产生式能够生成待搜索的右侧符号，或者说待搜索的右侧符号组合是否能够规约成为一个非终结符。为了加速产生式的搜索过程，程序依照右侧符号也就是搜索键，为输入的产生式进行排序，同时使用产生式索引。以下代码声明了产生式索引的结构体。

```
1 struct BeginAndNum
```

```

2 {
3     int begin;
4     unsigned num;
5 };

```

产生式索引结构体声明

该结构体用于存储开始索引和数量。**begin** 表示目标产生式的开始索引，**num** 表示数量。定义了两个该类型的数组变量，用于存放产生式索引。

```

1 BeginAndNum vnIndex [MAX_VN_NUM][MAX_VN_NUM];
2 BeginAndNum vtIndex [MAX_VT_NUM];

```

产生式索引变量定义

vnIndex 用于存储二元产生式的索引信息，指出其在 **production2** 数组中的位置，当需要查找 **child1** 和 **child2** 能生成哪些非终结符时，可以直接通过 **vnIndex** 快速定位到这些产生式，而不需要遍历整个产生式数组。**vtIndex** 用于存储一元产生式的索引信息，指出其在 **production1** 数组中的位置，当需要查找某个终结符 **t** 能生成哪些非终结符时，可以直接通过 **vtIndex** 快速定位到这些产生式。

4. 语法子树

程序声明以下结构体，用以存储子树根节点的非终结符，以及该子树可能的结构数量。需要注意的是，我们不保存完整的子树结构，仅仅记录数量。

```

1 struct SubTree
2 {
3     int root;
4     unsigned num;
5 } subTreeTable [MAX_STRING_LENGTH][MAX_STRING_LENGTH][MAX_VN_NUM];

```

子树声明与定义

其中 **root** 表示子树的根节点（非终结符），**num** 表示子树的数量。同时定义的 **subTreeTable** 数组中保存了输入字符串的子串的对应语法子树，具体来说 **subTreeTable[i][j]** 表示下标范围 **[i, j]**（从 0 开始）的子串能够规约形成的非终结符及对应子树的数量。

（二）算法理解

1. 算法核心过程

下图即为 CYK 算法伪代码。

```
function CKY-PARSE(words, grammar) returns table
```

```
  for j ← from 1 to LENGTH(words) do
```

```
    table[j - 1, j] ← {A | A → words[j] ∈ grammar }
```

```
    for i ← from j - 2 downto 0 do
```

```
      for k ← i + 1 to j - 1 do
```

```
        table[i, j] ← table[i, j] ∪
```

```
          {A | A → BC ∈ grammar,
```

```
            B ∈ table[i, k],
```

```
            C ∈ table[k, j] }
```

图 1 CYK 算法伪代码

对于这份伪代码， $table[i, j]$ 中保存的即为下标范围 $[i, j)$ （从 0 开始）的子串所能规约到的非终结符。

在外层循环中， j 从 1 到 $LENGTH(words)$ ，初始化每个单字符子串的位置。对于每个字符 $words[j]$ ，如果存在产生式 $A := words[j]$ ，则将 A 放入 $table[j-1, j]$ 中，表明第 j （从 1 开始）个字符，能够被规约成哪些非终结符。在中层循环中， i 从 $j-2$ 递减到 0，表明对于每一列，我们从下向上进行处理。在内层循环中，对于每个可能的起始位置 i ，遍历所有可能的划分点 k ，检查是否存在产生式 $A := BC$ ，其中 B 是 $table[i, k]$ 中的一个非终结符， C 是 $table[k, j]$ 中的一个非终结符。如果存在，则将 A 添加到 $table[i, j]$ 中。下图是示例的文法、输入串和算法执行结果。

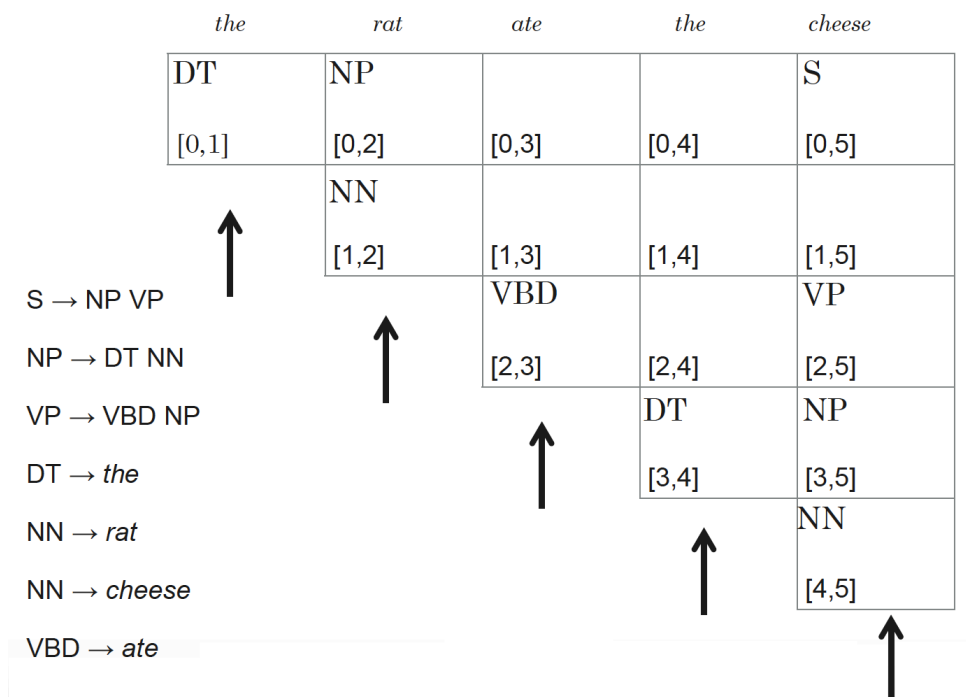


图 2 CYK 算法执行示例

对于这份伪代码，不难看出，其对于 table 的求解顺序是从左向右，自下而上的。也就是说对于这个上三角的 table，算法首先求解最左侧的列，在一列求解完毕后求解右侧的列。在每一列内，算法首先求解下面的项，然后逐个向上求解。

2. 数据依赖

由于我们要设计并行算法，因此必须考虑数据依赖问题。对于上述伪代码给出的 CYK 算法方案，一个 table 项目的求解依赖于其左侧的项目和下方的项目。在求解某个项目时，必须保证其左侧和下方的项目已经全部求解完毕。

（三）算法实现

提供的文件 serial_CYK.cpp 中提供了一个 CYK 算法的串行实现，这里对其进行梳理与解读。

1. 主要执行流程

在完成输入后，程序首先对 production1 数组进行排序，并建立索引 vtIndex，同时初始化表格 subTreeTable，填写其对角线上的元素，即完成对单个终结符的分析。然后对 production2 数组排序，并建立索引 vnIndex。

然后进入算法的核心部分，也就是子树生成的阶段。在该阶段中，对于表格 subTreeTable 中的每一个项目，也就是每一个子串的子树信息，程序首先将其存储到一个缓冲区中。在每个项目求解完毕后，将其排序，再保存到表格中。关于各个项目的求解

顺序，程序实现与上述伪代码不同。在程序中，最外层循环遍历子串长度 `len`，从 2 到 `string_length`。第二层循环遍历子串的左起始点 `left`，从 0 到 `string_length-len`，可以通过左起始点和长度唯一的标识子串，也就是说已经定位到了表格中待求解的项目。第三层循环遍历所有可能的子串划分点 `right`，或者说是右子串的起始点，从 `left+1` 到 `left+len`，对于刚刚唯一确定的子串，我们需要将其划分为两部分，寻找是否有二元产生式与之对应。至此我们已经确定了子串的划分形式，第四层循环遍历左子串在表格中保存的信息，也就是遍历左子串对应的子树，而第五层循环也就是最内层循环，则遍历右子串对应的子树，最后查询产生式，寻找其是否能够规约成为一个非终结符。

最后，程序根据表格信息 `subTreeTable[0][string_length-1]` 进行输出

2. 输入

下发的作业说明文档中也提供了对于输入的说明。

从 “input.txt” 读入。第一行是一个整型，表示非终结符的数量 `vn_num`；第二行是一个整型，表示“右侧是两个非终结符的产生式”的数量 `production2_num`，即代码中的 `Production2` 类型的数量；接下来 `production2_num` 行，每行是一个 `Production2` 类型，非终结符的格式是 “<%d>”，数字范围是 0 到 (`production2_num-1`)，“<0>” 表示开始符号；然后下一行是一个整型，表示“右侧是一个终结符的产生式”的数量 `production1_num`，即代码中的 `Production1` 类型的数量；接下来 `production1_num` 行，每行是一个 `Production1` 类型，终结符的格式是 “%c”，范围是所有可打印的 ASCII 字符；然后下一行是一个整型，表示被测试的字符串的长度；最后一行是被测试的字符串。

结合代码不难发现，我们允许产生式是无序的，一个非终结符可以生成多种串，同一种串也能够出现在不同产生式的右侧。但是终结符必须是单个可见字符。

二、并行程序设计与实现

（一）并行程序设计

在前面的章节中，我们讨论到，提供的串行的代码在求解表格中项目时，是逐斜线求解的。在初始化的时候求解上三角阵左下方（从左上 `[0, 0]` 至右下）的对角线，然后向右上方移动。这种求解顺序非常适合并行程序，因为对于每一轮，也就是每次的一条斜线，也就是最外层循环的每一次执行，其中所有待求解项目依赖的数据都已经准备好了。而伪代码的求解顺序则不允许在同一个层次上进行并行化，因为伪代码中的求解顺序是从左至右，从下至上的。每个最外层循环的执行包含一系列的项目，而上方的项目依赖下方的项目，不能在这个层次上并行。但是仍可以考虑在每个项目的求解之中并行，

但这可能需要处理数据冲突。

因此，对于提供的串行代码，我就计划按上面提到的方案，串行化的执行最外层的循环，按顺序依次求解不同长度子串能够归约到的非终结符；对于每个给定的子串长度，并行化的求解不同起始位置的子串。结合表格形象的说明的话，就是并行求解一条斜线上的各个项目，然后在这一条斜线上所有的项目求解完毕后，再并行的求解下一条斜线。

（二）并行程序实现

实现上述并行化设计的核心就是并行执行第二层循环，可以采用如下方案。

```
1 // 最外层循环，遍历子串长度
2 for (int len = 2; len <= string_length; len++)
3 {
4     // 第二层循环，遍历子串起点
5     # pragma omp parallel for num_threads(thread_count)
6     for (int left = 0; left <= string_length - len; left++)
7     {
```

并行代码实现方案

在上述代码局部中，我在第二层循环外使用了编译器指令 `pragma omp parallel for`，创建多个线程并行的执行第二层循环。但在课上我们提到，这样可能会导致频繁的线程 `create` 和 `join`，带来额外的开销，因此，我使用以下这种实现方案。

```
1 // 最外层循环，遍历子串长度
2 # pragma omp parallel num_threads(thread_count)
3 for (int len = 2; len <= string_length; len++)
4 {
5     // 第二层循环，遍历子串起点
6     # pragma omp for
7     for (int left = 0; left <= string_length - len; left++)
8     {
```

优化后的并行代码实现方案

在这段代码中，我在进入最外层循环前，就创建了并行区域，完成了多个线程的创建，避免了在循环中频繁的创建和销毁线程。

三、实验

在完成了代码的并行化后，通过实验来观察并行化对程序运行时间带来的变化。

（一）实验环境

程序在我个人的家用服务器上运行，处理器为英特尔® 酷睿™ i7-12700，最大睿频频率 4.90 GHz，8 个性能核和 4 个能效核，共 20 个线程，双通道内存共 64G，DDR4 3200MHz。操作系统为 Ubuntu 23.10 Server，直接运行在物理机上。程序直接运行在宿主操作系统上，而非运行在容器中。

在进行实验时，宿主机上所有的容器均已停止，但仍有大量进程运行，只不过它们几乎不使用 CPU。

（二）实验组别

1. 串行执行

串行代码处理输入文件“input2.txt”的执行时间，作为对比的基准。之后的组中都使用该文件作为输入。

2. 并行区域位于外层循环内

使用在“并行代码实现方案”中展示的实现方式，在外层循环内创建和销毁线程。

3. 默认调度策略

从该组开始，包括动态调度策略、默认引导式调度的各组，均使用“优化后的并行代码实现方案”中展示的实现方式，在外层循环外创建并行区域，在并行区域内再并行化第二层 for 循环。

这里使用默认的调度策略，即静态调度，同时不指定 chunk_size 块大小。此时 OpenMP 会将 for 循环按其下标均匀的分配给各个线程执行。

4. 动态调度策略

循环迭代被分成大小相等的块，并动态分配给线程。当一个线程完成一个块时，它会获取下一个块。在该组中，使用了默认（1），4，16 三种不同的块大小进行测试。

```
1 // 最外层循环，遍历子串长度
2 # pragma omp parallel num_threads(thread_count)
3 for (int len = 2; len <= string_length; len++)
4 {
5     // 第二层循环，遍历子串起点
6     # pragma omp for schedule(dynamic, 4)
7     for (int left = 0; left <= string_length - len; left++)
8     {
```

块大小为 4 的动态调度策略

5. 默认引导式调度

类似于 `dynamic`，但块的大小逐渐减小。初始块较大，后续块逐渐变小，直到达到指定的 `chunk_size`。这里没有指定块大小，默认为 1。

```
1 // 最外层循环，遍历子串长度
2 # pragma omp parallel num_threads(thread_count)
3 for (int len = 2; len <= string_length; len++)
4 {
5     // 第二层循环，遍历子串起点
6     # pragma omp for schedule(guided)
7     for (int left = 0; left <= string_length - len; left++)
8     {
```

默认引导式调度

(三) 实验数据记录表

单位为微秒。

表 1 实验数据记录表

组别	数据 1	数据 2	数据 3	数据 4	数据 5	平均
串行程序	47290223	47429547	47243947	47294040	46829509	47217453.2
并行区域位于外层循环内（线程数 1）	50109798	50566555	49595740	49606736	49577341	49891234
并行区域位于外层循环内（线程数 2）	25560577	25904316	25632841	25627501	25563017	25657650.4
并行区域位于外层循环内（线程数 4）	13189456	13153158	13225255	13200646	13222267	13198156.4
并行区域位于外层循环内（线程数 8）	7052946	7013102	6991525	6971041	6980119	7001746.6
并行区域位于外层循环内（线程数 16）	6486875	6449169	6482928	6435375	6428611	6456591.6
并行区域位于外层循环内（线程数 32）	6276551	6163628	6202596	6217395	6172942	6206622.4
默认调度策略（线程数 1）	50216643	49604925	49733300	49642874	51016855	50042919.4
默认调度策略（线程数 2）	26177365	25596754	25536314	26180371	25952469	25888654.6
默认调度策略（线程数 4）	13165393	13158068	13243478	13167513	13173469	13181584.2
默认调度策略（线程数 8）	6953577	6950707	6938714	7037207	6992164	6974473.8
默认调度策略（线程数 16）	6492385	6503187	6465681	6573176	6604548	6527795.4
默认调度策略（线程数 32）	5660697	5573832	5564571	5533691	5582940	5583146.2
默认动态调度策略（线程数 1）	48919220	48900132	48842425	48850797	49992504	49101015.6
默认动态调度策略（线程数 2）	25090084	25113430	25393433	25083938	25355806	25207338.2
默认动态调度策略（线程数 4）	13005568	12914429	12963346	12949802	13041973	12975023.6
默认动态调度策略（线程数 8）	6842167	6817261	6844541	6839818	6843544	6837466.2
默认动态调度策略（线程数 16）	5053150	5010014	5025454	5047820	5157809	5058849.4
默认动态调度策略（线程数 32）	4857386	4862722	4850516	4938866	4979993	4897896.6
块大小为 4 的动态调度策略（线程数 1）	48874635	49364019	50162451	48816863	49039505	49251494.6
块大小为 4 的动态调度策略（线程数 2）	25338184	25547935	25259233	25287723	25275920	25341799
块大小为 4 的动态调度策略（线程数 4）	13147614	13201528	13143122	13191606	13179603	13172694.6
块大小为 4 的动态调度策略（线程数 8）	7125111	7109418	7110982	7117465	7107781	7114151.4
块大小为 4 的动态调度策略（线程数 16）	5559692	5608104	5565223	5554065	5563028	5570022.4
块大小为 4 的动态调度策略（线程数 32）	5422694	5428728	5392114	5423129	5416209	5416574.8
块大小为 16 的动态调度策略（线程数 1）	50174676	49163463	49024093	48977602	49081626	49284292
块大小为 16 的动态调度策略（线程数 2）	25758806	25876862	25795849	25953303	25897297	25856423.4
块大小为 16 的动态调度策略（线程数 4）	14046048	13987214	14062960	14085373	14095462	14055411.4
块大小为 16 的动态调度策略（线程数 8）	8243259	8249818	8245257	8234858	8250609	8244760.2
块大小为 16 的动态调度策略（线程数 16）	7620321	7590981	7636432	7656990	7578567	7616658.2
块大小为 16 的动态调度策略（线程数 32）	7209361	7128360	7075783	7158845	7061271	7126724
默认引导式调度（线程数 1）	49023811	50161119	49547991	48971514	48859851	49312857.2
默认引导式调度（线程数 2）	25124326	25300796	25117492	25116556	25275798	25186993.6
默认引导式调度（线程数 4）	12911455	12928859	13018339	12901014	12905975	12933128.4
默认引导式调度（线程数 8）	6831869	6839402	6809426	6824238	6812803	6823547.6
默认引导式调度（线程数 16）	5738434	5683263	5690302	5838473	5854713	5761037
默认引导式调度（线程数 32）	5344789	5286660	5264765	5366594	5775076	5407576.8

四、问题的回答

（一）如何将任务进行分块的？

这个问题在“并行程序设计”一节中进行了详细解释。

（二）如何提升运行速度与增强程序扩展性？

串行化本身就可以提升运行速度，可以通过调整数据的划分，并行化的实现方案，线程间的调度算法等等来进一步优化。此外，也可以使用更强大的处理器，进一步优化算法等来提高运行速度，如代码中已经实现的产生式排序和索引。

目前的程序仅仅支持上下文无关文法，但这是 CYK 算法本身决定的，暂且不讨论。

在“算法实现”一节中也讨论了程序输入的约束，目前终结符仅仅支持单个字符，可以对其进行略微修改，使用长整型等表示终结符。这样我们既可以将其映射到字符，直接表示终结符，也可以令其表示词素，使得程序可以与词法分析器对接，支持更加复杂的文法。

（三）如何测量程序运行时间？

我使用 chrono，在程序内部添加代码，以输出主要部分的执行时间。

```
1 #include <iostream>
2 #include <chrono>
3
4 using namespace std;
5 using namespace std::chrono;
6
7 int main() {
8     /* 读取输入文件 */
9
10    // 记录开始时间
11    auto start = high_resolution_clock::now();
12
13    /* 建立产生式索引 */
14    /* 执行CYK算法 */
15
16    auto stop = high_resolution_clock::now();
17    auto duration = duration_cast<microseconds>(stop - start);
18
19    /* 输出CYK算法结果 */
20
21    // 输出运行时间
22    cout << "time: " << duration.count() << " us" << endl;
23
24    return 0;
25 }
```

时间记录代码

（四）比较图

以下是不同组别方案的耗时时间比较图，单位是微秒。具体各个组内的实现方案在“实验组别”一节中进行了介绍。

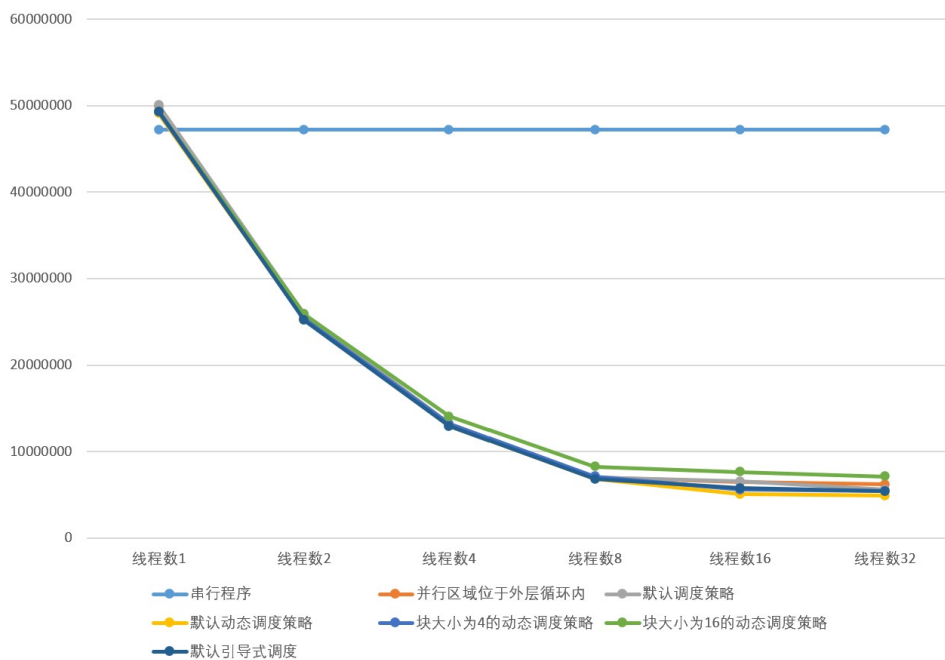


图3 耗时比较折线图

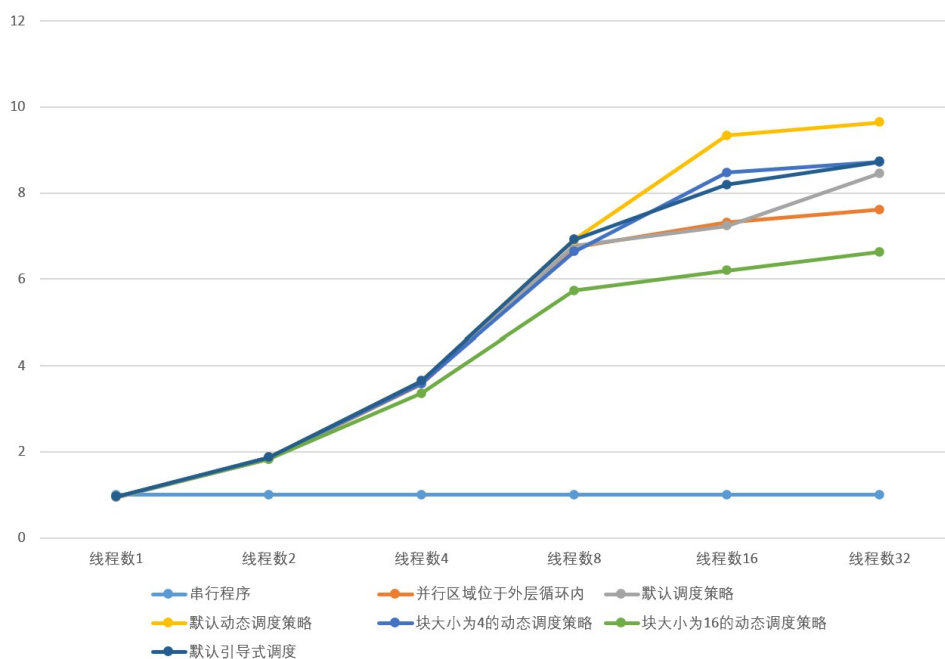


图4 加速比比较折线图

（五）加速比变化的趋势

通过观察图3和图4，可以发现随着线程数的增加，并行策略的加速比呈现上升趋势，说明并行化在一定程度上提高了程序的执行效率。在低线程数（如线程数2和线程数4）时，加速比增长较快，体现了并行化的初步优势。在较高线程数（如线程数16和线程数32）时，加速比的增长开始放缓，甚至出现下降的趋势，这可能是由于并行化带

来的开销（如线程管理、同步等）超过了计算的加速收益，以及仅有 20 个物理线程的限制。

通过对比“并行区域位于外层循环内”和“默认调度策略”两条线，可以发现似乎将线程的创建和销毁移动到最外层循环外并没有明显的作用，这可能是由于最外层的循环并不会执行非常多次，且线程的创建和销毁相比于任务本身，花费很小导致的。

默认的动态调度策略表现比默认调度策略好，可以说明任务的花费不太均匀，动态分配任务的收益大于调度的额外开销。随着块大小的增大，动态分配策略表现的越来越差，这很可能是较大的任务块使得对于各个线程任务分配的不均匀，尤其是当算法执行到后面时，每个循环中需要并行的任务数量（下标）并不多，较大的块大小甚至可能导致有大量线程闲置。

引导式调度策略表现一般，进一步印证对于该任务，调度的花费其实并不高，小于任务分配不均带来的额外开销，不需要开始使用大块来减轻调度任务，更精细的调度策略可能是更好的选择。

（六）最佳方案

最佳的方案是使用默认的动态调度策略，已经在上一节中进行了分析。